FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Villaraza, John Patrick B.
Section:	AN41
Professor:	Crisola G. Tan
Date:	October 6, 2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects

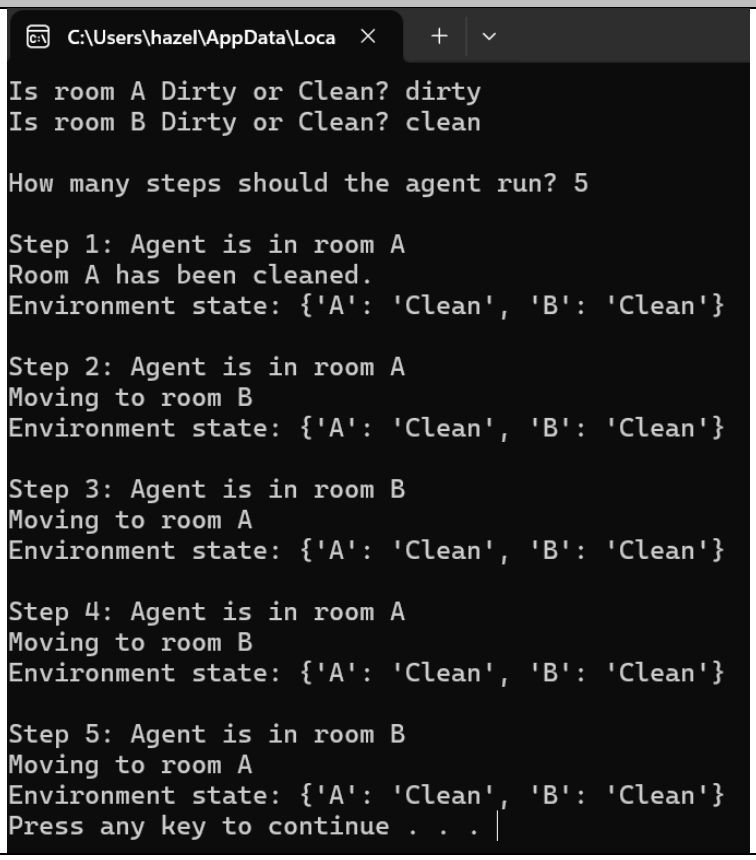
- If-Else statements
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

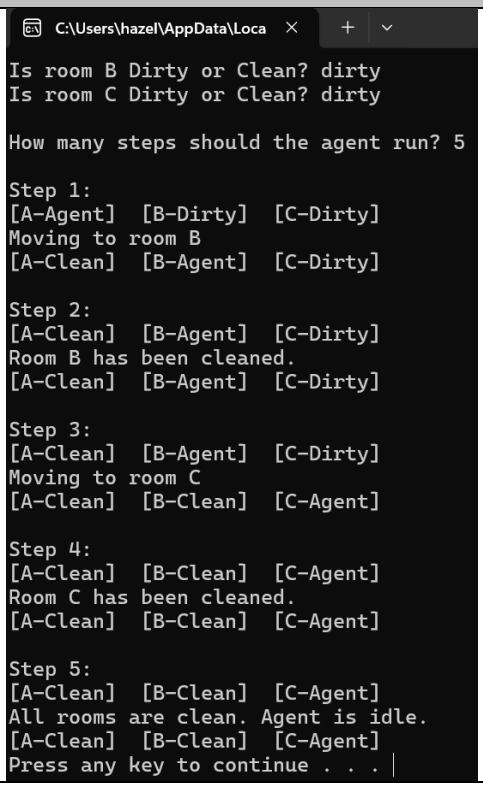
Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

```
class VacuumAgent:
    def __init__(self, initial_location='A'):
        self.location = initial_location

    def move(self):
```

```
        """Switches the agent's location between room A and room B."""
        self.location = 'B' if self.location == 'A' else 'A'

    def perceive_and_act(self, environment):
        """
        Executes rule-based logic:
        Rule 1: If the current room is dirty, clean it.
        Rule 2: If the current room is clean, move to the other room.
        """
        current_status = environment[self.location]

        if is_dirty(environment, self.location):
            clean_room(environment, self.location)
            print(f"Room {self.location} has been cleaned.")
        else:
            other_room = 'B' if self.location == 'A' else 'A'
            print(f"Moving to room {other_room}")
            self.move()


def is_dirty(environment, room):
    """Checks if a given room is dirty."""
    return environment[room].strip().capitalize() == 'Dirty'


def clean_room(environment, room):
    """Cleans a given room in the environment."""
    environment[room] = 'Clean'


def main():
    # Task 1: Set Up the Environment
    room_a = input("Is room A Dirty or Clean? ").strip().capitalize()
    room_b = input("Is room B Dirty or Clean? ").strip().capitalize()

    environment = {
        'A': room_a,
        'B': room_b
    }

    # Task 2 & 3: Initialize Agent and Run Simulation
    agent = VacuumAgent(initial_location='A')
```

```
    steps = int(input("\nHow many steps should the agent run? "))
    print()

    for step in range(1, steps + 1):
        print(f"Step {step}: Agent is in room {agent.location}")
        agent.perceive_and_act(environment)
        print(f"Environment state: {environment}\n")


if __name__ == "__main__":
    main()
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. What rules does the agent follow?
   - If the room is dirty, clean it
   - If the room is clean, move to other room
2. How does the agent decide when to clean and when to move?
   - Agent checks the state of the room whether the room is dirty or clean. If dirty, agent cleans the room. If clean, agent moves to other room.
3. What happens if both rooms are already clean at the start?
   - Rule 1 will be skipped
4. How would you modify the agent to handle more than two rooms?
   - Expand the Environment: Represent the rooms using a larger collection, such as a list of room names (e.g., ['A', 'B', 'C', 'D']) or an expanded dictionary.
   - Update Movement Logic: Replace the simple two-way toggle (A \leftrightarrow B) with a systematic traversal method, such as:
   - Sequential/Cyclic Indexing: Increment the current index to move to the next adjacent room (e.g., A \rightarrow B \rightarrow C \rightarrow A).
   - Directional Movement: Move left/right based on boundary limits.
   - Search-based / Goal-driven: Pass a list of uncleaned room coordinates or names so the agent moves toward the nearest dirty room.